Splunk SIEM Project

Analyzing SSH Log Files Using Splunk SIEM

Hassen Hannachi

SOC Analyst Lab Series · Blue Team

March 31, 2026

Introduction

SSH (Secure Shell) log files contain valuable information about remote access to servers, including login attempts, commands executed, and session details. Analyzing SSH logs using Splunk SIEM enables security professionals to monitor access to critical systems, detect anomalies, and identify potential security threats across the network.

Project Overview: Sample SSH log files will be ingested into the Splunk SIEM platform and analyzed to extract actionable insights into SSH activity. This includes monitoring remote connections, authentication behaviour, and user activity. By correlating these logs in Splunk, security teams can detect suspicious or unauthorized access and strengthen overall threat-detection capabilities.

Prerequisites: Before starting this project, ensure the following are in place:

- A Splunk instance is installed and configured.
- SSH log data sources are configured to forward logs to Splunk.

SSH Log Field	Description
`ts_epoch`	Timestamp in Unix epoch format (with decimals)
`uid`	Unique session identifier
`src_ip`	Source (client) IP address
`src_port`	Source port of the SSH connection
`dst_ip`	Destination (server) IP address
`dst_port`	Destination port (typically port 22)
`result`	Connection outcome: `failure` or other status
`direction`	Connection direction (e.g., INBOUND)
`client_version`	SSH client software version string
`server_version`	SSH server software version string
`status`	Extracted connection status field

Part 1 — Uploading SSH Log Files to Splunk

1. Prepare Sample SSH Log Files

- Obtain sample SSH log files in a suitable format (e.g., plain text or compressed `.gz` files). A public sample set is available at: `https://www.secrepo.com/maccdc2012/ssh.log.gz`
- Ensure the log files contain relevant SSH events including timestamps, source IP addresses, usernames, and actions (login, logout, etc.).
- Save the sample log files in a directory accessible by the Splunk instance.

2. Upload Log Files to Splunk

- Log in to the Splunk web interface.
- Navigate to **Settings** → **Add Data**.

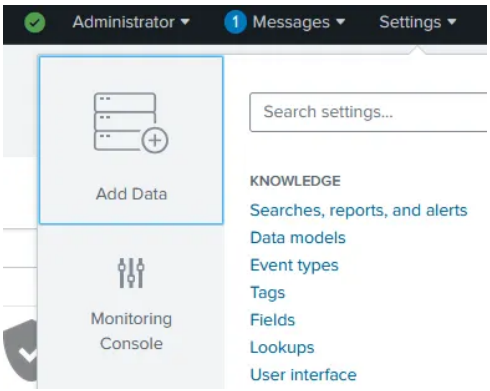

Figure 1: Splunk — Settings → Add Data menu

- Select **Upload** as the data input method.

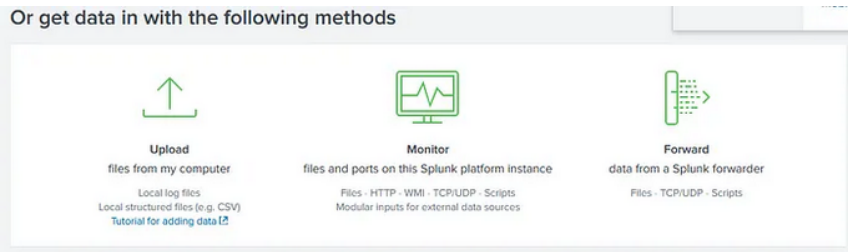

Figure 2: Selecting the Upload data input method

3. Choose the Log File

Click **Select File**, choose the sample SSH log file prepared in Step 1, then click **Next** in the top-right corner.

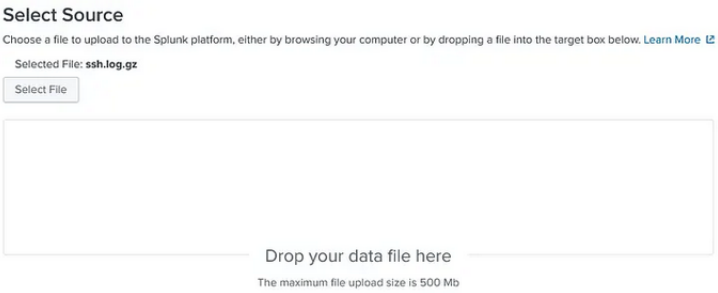

Figure 3: Selecting the sample SSH log file for upload

4. Set the Source Type

In the **Set Source Type** section, specify the source type for the uploaded log file. Choose an appropriate source type for SSH logs — for example, `syslog` — or define a custom source type (e.g., `sshlog`).

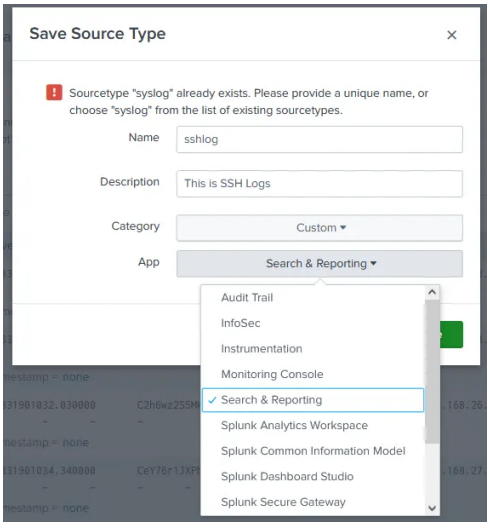

Figure 4: Configuring the source type for the SSH log file

5. Review Settings

Review the remaining settings including **index**, **host**, and **sourcetype**. Ensure all values match the sample SSH log file before proceeding.

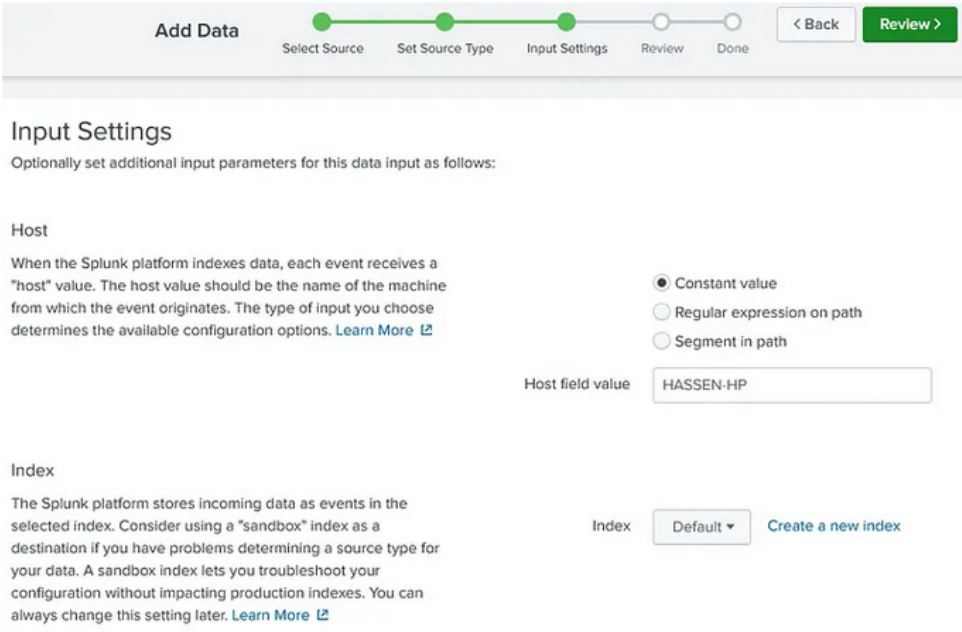

Figure 5: Reviewing index, host, and sourcetype settings

6. Submit the Upload

- Click the **Review** button and inspect all settings one final time.
- Click **Submit** to upload the SSH log file to Splunk.

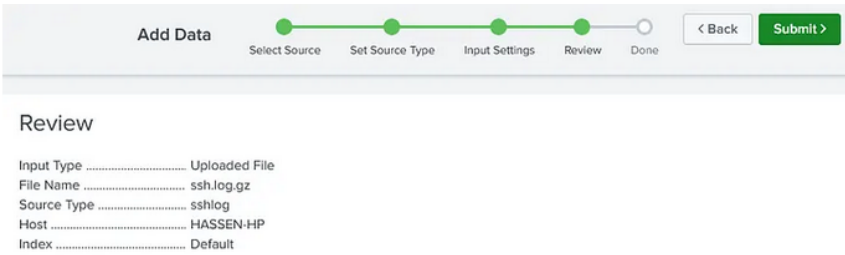

Figure 6: Submitting the SSH log file upload

7. Verify the Upload

Navigate to the Splunk search bar and run a query to confirm that the uploaded SSH events are visible:

```
index=_* OR index=* sourcetype=sshlog
```

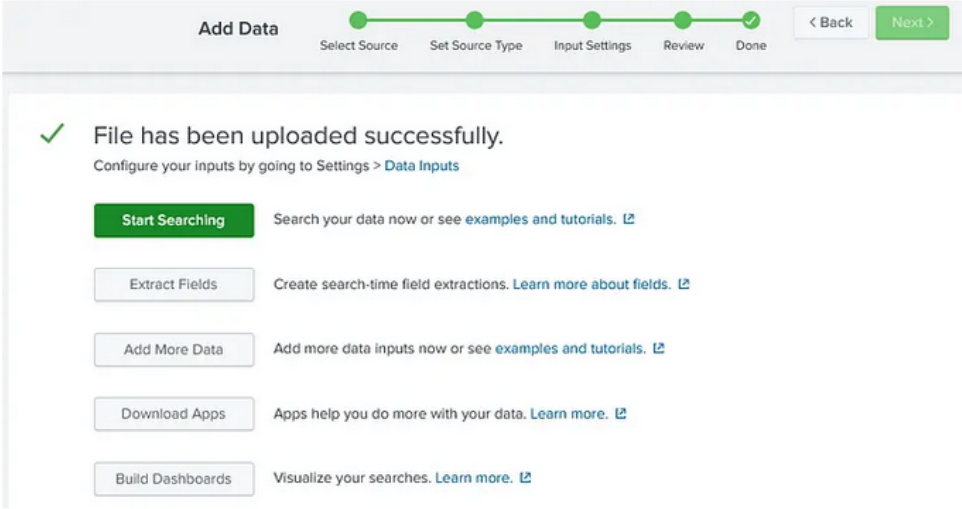

Figure 7: Verifying that uploaded SSH events appear in search results

8. Parse and Extract Fields

Navigate to **Extract New Fields** in the Splunk interface to define custom field extractions from the raw SSH log data.

Figure 8: Opening the Extract New Fields panel

Select any event from the list and click **Next**.

Figure 9: Selecting a sample event for field extraction

Select **Regular Expression** as the extraction method (Delimiters can also be used), then click **Next**.

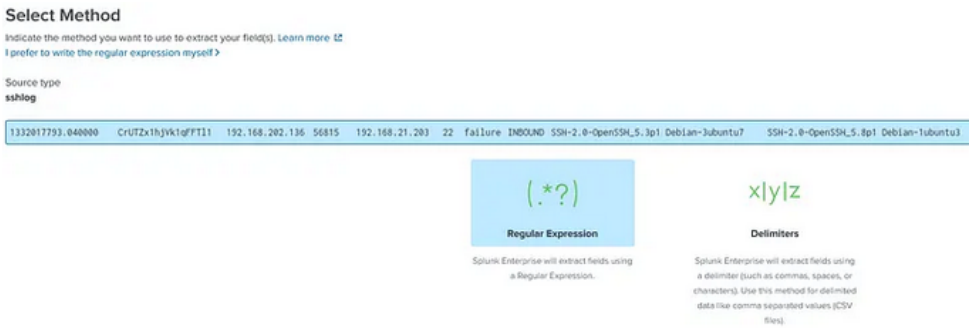

Figure 10: Choosing Regular Expression as the extraction method

Extract the following fields one by one by clicking on the relevant value in the event preview and assigning a field name:

Value Selected	Field Name	Description
`192.168.202.136`	`src_ip`	Client (source) machine IP address
`192.168.202.136`	`dest_ip`	Destination machine IP address
`56815`	`src_port`	Source port of the connection
`22`	`dest_port`	Destination port (SSH default)
`failure`	`status`	Connection status

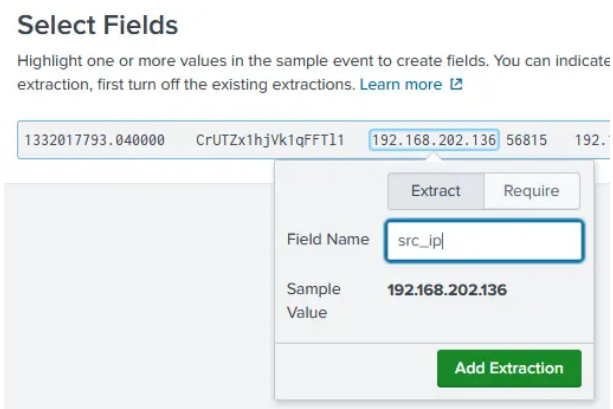

Figure 11: Extracting the src_ip field (client address)

After all fields are validated and saved, click **Finish** to complete the extraction.

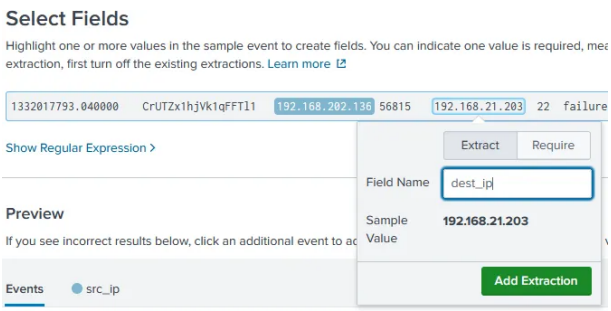

Figure 12: xtracting the dest_ip field (destination address)

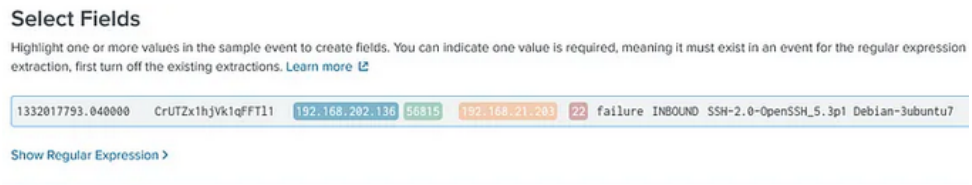

Figure 13: Extracting src_port (56815) and dest_port (22)

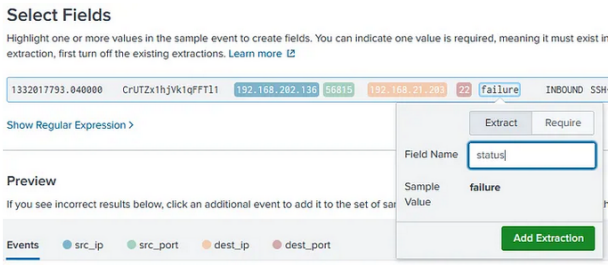

Figure 14: Extracting the status field (failure)

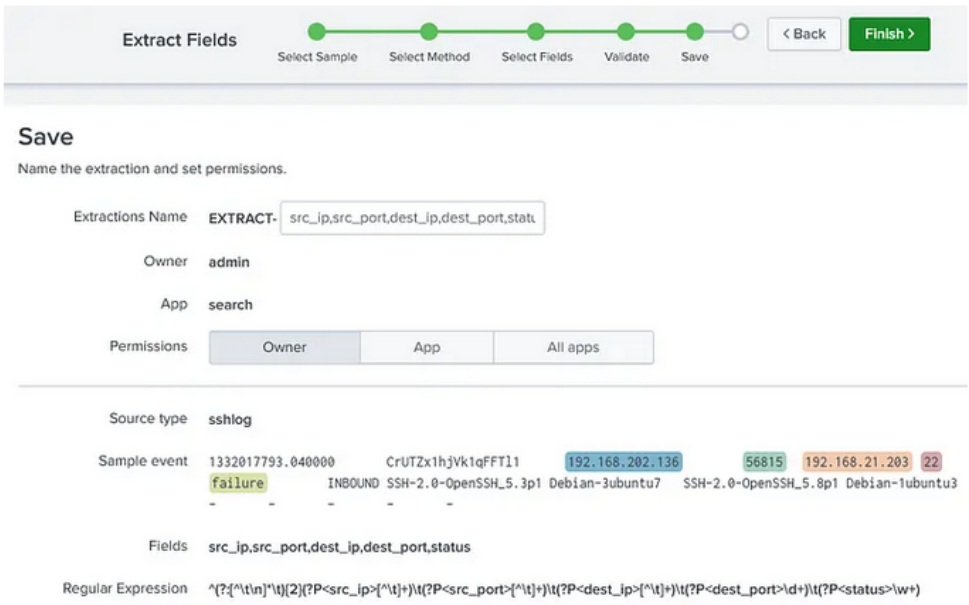

Figure 15: Validating and saving all extracted fields

Click **Explore the fields I just created in Search** to navigate back to the search

interface.

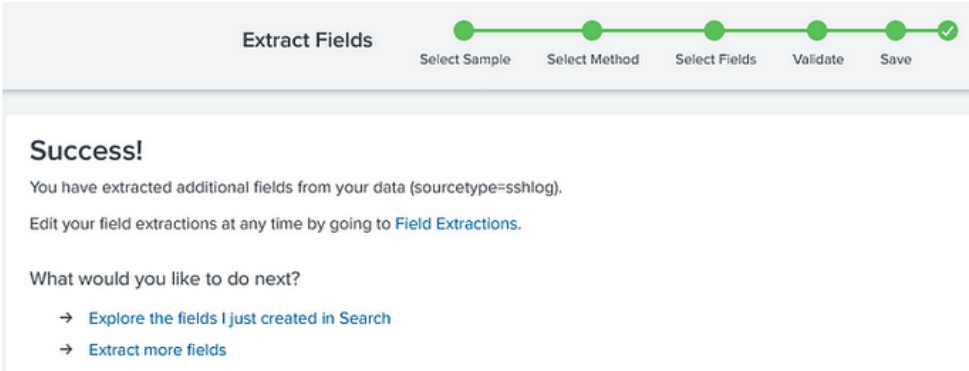

Figure 16: Navigating to search after field extraction

Result: The **Interesting Fields** panel now contains all the newly extracted fields:
`dest_ip` `dest_port` `status` `src_ip` `src_port`

Figure 17: Interesting Fields panel showing all extracted SSH fields

Part 2 — Analyzing SSH Log Files in Splunk

9. Search for SSH Events

Open the Splunk search bar and enter the following query to retrieve all SSH events from the uploaded log source:

```
index=_* OR index=* sourcetype=sshlog
```

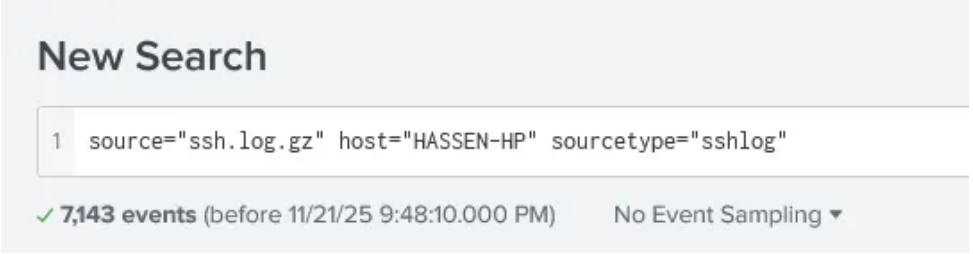

Figure 18: Retrieving all SSH events from the Splunk index

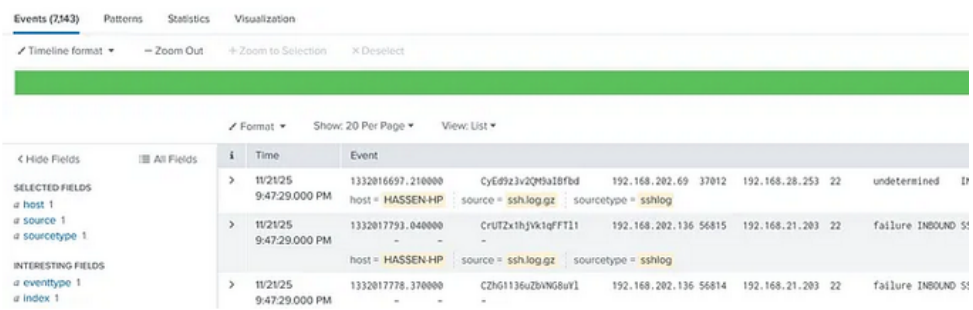

Figure 19: SSH events displayed in the Splunk search results view

10. Extract Relevant Fields with Rex

Use the `rex` command with a regular expression to extract structured fields from the raw SSH log data, enabling richer correlation and analysis:

```
index=_* OR index=* sourcetype=sshlog
| rex field=_raw
  "(?<ts_epoch>\d+\.\d+)\s+(?<uid>\S+)\s+(?<src_ip>\d+\.\d+\.\d+\.\d+)\s+
   (?<src_port>\d+)\s+(?<dst_ip>\d+\.\d+\.\d+\.\d+)\s+(?<dst_port>\d+)\s+
   (?<r>\S+)\s+(?<direction>\S+)\s+(?<client_version>\S+)\s+
   (?<server_version>\S+)"
| eval timestamp = strftime(ts_epoch, "%Y-%m-%d %H:%M:%S")
| stats
    count(eval(result="failure"))  AS failed_attempts,
    count(eval(result!="failure")) AS non_failures,
    dc(src_ip)                     AS unique_source_ips,
    dc(uid)                        AS unique_sessions
  BY direction, client_version, server_version
| sort -failed_attempts
```

Explanation of Key Query Parts:

- **`rex` extraction** — Parses each raw log line into named fields: `ts_epoch` (epoch timestamp), `uid` (session ID), `src_ip`/`src_port` (client), `dst_ip`/`dst_port` (server), `result` (failure or success), `direction`, `client_version`, and `server_version`.
- **`eval timestamp`** — Converts the epoch field into a human-readable timestamp for reporting purposes.
- **`stats`** — Counts failed vs. non-failed attempts, and computes distinct source IPs (`dc(src_ip)`) and unique sessions (`dc(uid)`) to help identify many different attackers.
- **`BY direction, client_version, server_version`** — Groups results by connection direction and SSH software versions, useful for spotting patterns such as all failures originating from outdated SSH clients.
- **`sort -failed_attempts`** — Sorts results in descending order by failure count to surface the worst offenders first.

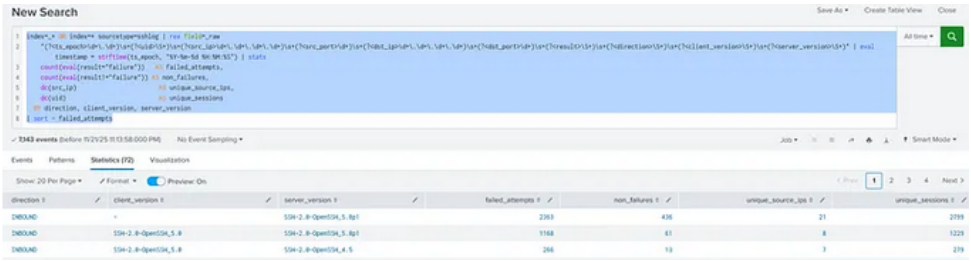

Figure 20: Rex command extracting structured fields from raw SSH log events

11. Analyze SSH Activity Patterns

11.1 Distribution of SSH Commands

Determine which commands are most frequently executed over SSH sessions:

```
index=_* OR index=* sourcetype=sshlog
| stats count by command
```

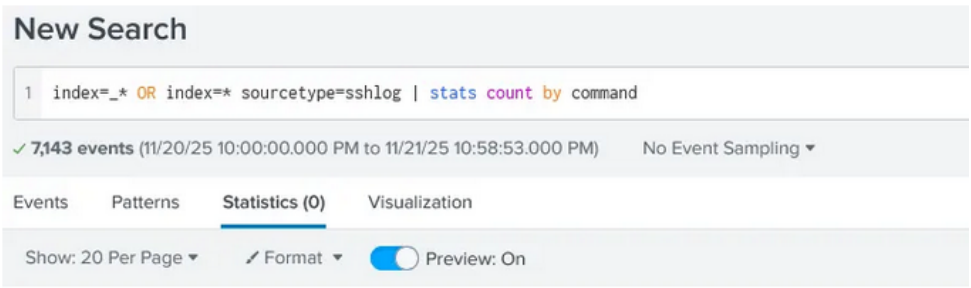

Figure 21: Count of SSH events grouped by executed command

11.2 Top Users and Source IPs

Identify the top usernames and source IP addresses accessing the SSH server:

```
index=_* OR index=* sourcetype=sshlog
| top limit=10 user src_ip
```

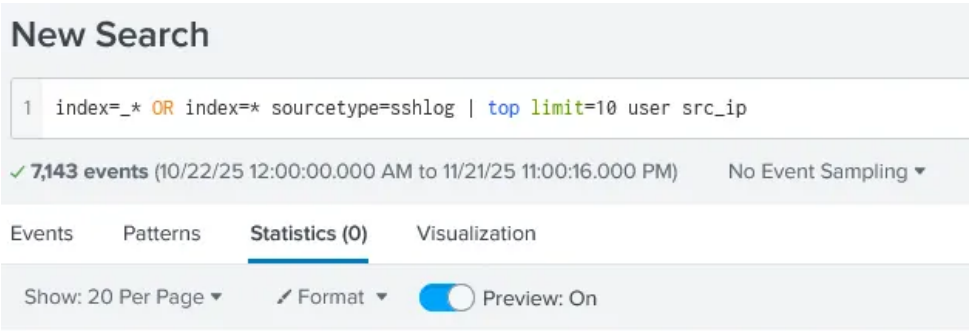

Figure 22: Top 10 users and source IPs accessing the SSH server

11.3 Top Attacking IPs

Rank source IPs by their event count and associated result to identify the most active attackers:

```
index=_* OR index=* sourcetype=sshlog
| rex field=_raw
  "^(?<ts_epoch>\d+\.\d+)\t(?<uid>\S+)\t(?<src_ip>[^\t]+)\t
   (?<src_port>\d+)\t(?<r>\S+)"
| stats count BY src_ip result
| sort -count
```

Figure 23: Top attacking IPs ranked by event count and result

11.4 Successful vs. Failed Login Attempts

Compare the volume of successful SSH connections against failed ones:

```
index=_* OR index=* sourcetype=sshlog
| rex field=_raw
  "^(?<ts_epoch>\d+\.\d+)\t(?<uid>\S+)\t(?<src_ip>[^\t]+)\t
   (?<src_port>\d+)\t(?<dst_ip>[^\t]+)\t(?<dst_port>\d+)\t(?<r>\S+)"
| stats count BY result
```

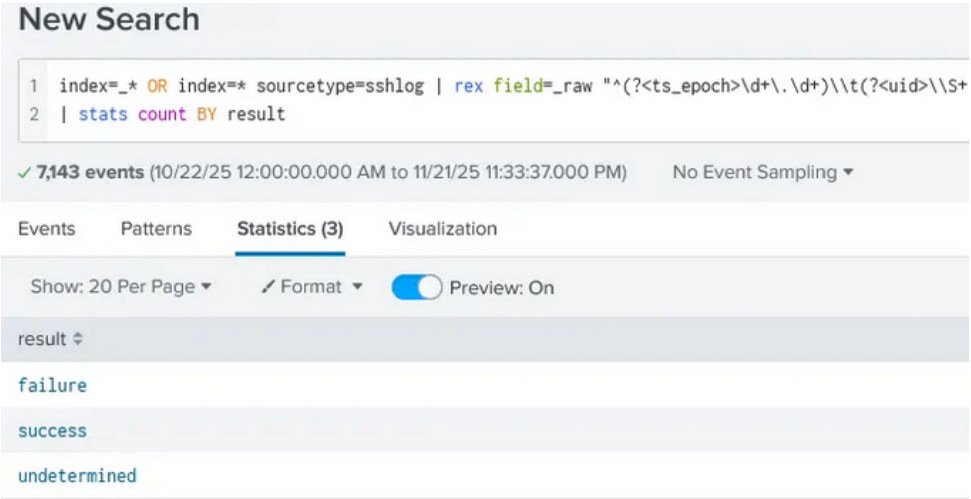

Figure 24: Comparison of successful vs. failed SSH login attempts

11.5 Time Chart of Failures

Visualise SSH failure events over time to identify attack spikes:

```
index=_* OR index=* sourcetype=sshlog
| rex field=_raw "^(?<ts_epoch>\d+\.\d+)\t(?<r>\S+)"
| eval _time = ts_epoch
| timechart span=5m count(eval(result="failure")) AS failures
```

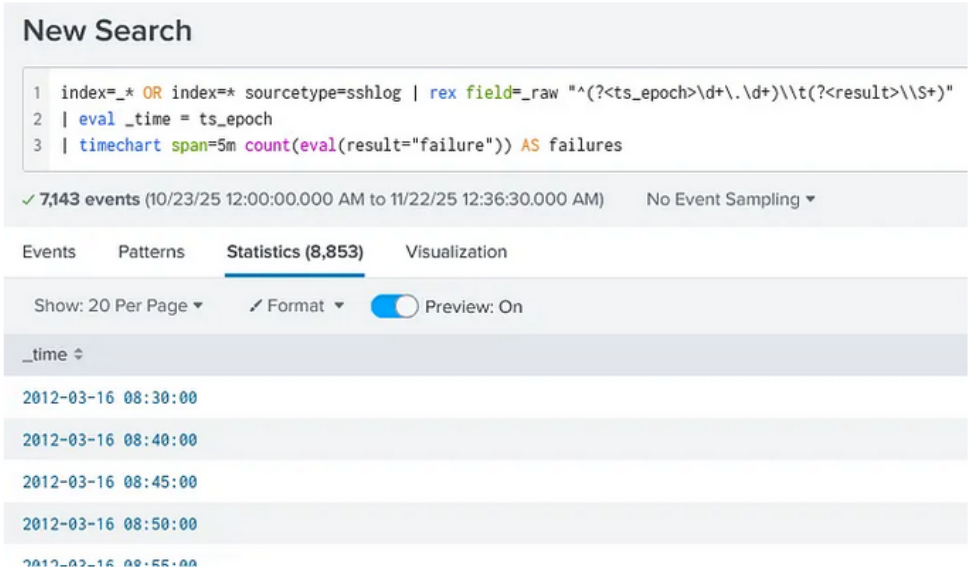

Figure 25: 5-minute time chart of SSH failure events

12. Detect Anomalies

12.1 Sudden Spikes in Login Attempts

Look for unusual hourly traffic patterns that may indicate automated scanning or brute-force campaigns:

```
index=_* OR index=* sourcetype=sshlog
| timechart span=1h count by _time
```

12.2 Failed Login Attempts Filter

Isolate all events where the action is recorded as failed:

```
index=_* OR index=* sourcetype=sshlog
| search action="failed"
```

12.3 Rare Source IPs — Low-Frequency Anomalies

Find source IPs that appear very rarely in SSH activity. Low-frequency connections can be highly suspicious, as they may represent stealthy reconnaissance or a one-off intrusion attempt:

```
index=_* OR index=* sourcetype=sshlog
| rex
  "^(?<ts_epoch>\d+\.\d+)\t(?<uid>\S+)\t(?<src_ip>\S+)\t
   (?<src_port>\d+)\t(?<dst_ip>\S+)\t(?<dst_port>\d+)\t(?<r>\S+)"
| stats count BY src_ip
| eventstats avg(count) AS avg_cnt stdev(count) AS sd_cnt
| where count < avg_cnt - (2 * sd_cnt) OR count < 3
| sort count
| rename count AS "Event Count"
```

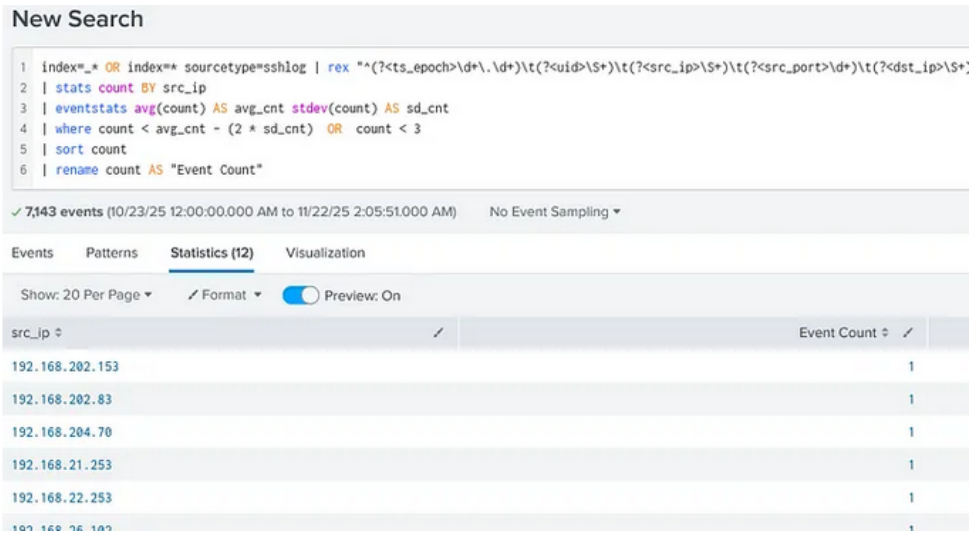

Figure 26: Source IPs with statistically low SSH event counts flagged as anomalous

12.4 Detect Brute-Force IPs by High Failure Rate

Flag source IPs that have generated more than 20 failed authentication attempts — a strong indicator of brute-force or credential-stuffing attacks:

```
index=_* OR index=* sourcetype=sshlog
| rex field=_raw
```

```
  "^(?<ts_epoch>\d+\.\d+)\t(?<uid>\S+)\t(?<src_ip>[^\t]+)\t(?<r>\S+)"
| stats count(eval(result="failure")) AS failures BY src_ip
| where failures > 20
| sort -failures
```

Figure 27: Source IPs exceeding 20 SSH failures flagged as potential brute-force attackers

Threat Indicator: Any source IP generating more than 20 failed SSH attempts in a short window should be investigated immediately and considered for blocking at the firewall or intrusion prevention system (IPS) level.

13. Monitor User Behaviour

13.1 Source IPs with Multiple Failed Attempts

Identify all source IPs that have accumulated more than three failed login attempts, which may indicate a targeted attack against specific accounts:

```
index=_* OR index=* sourcetype=sshlog
| rex
  "^(?<ts_epoch>\d+\.\d+)\t(?<uid>\S+)\t(?<src_ip>\S+)\t
   (?<src_port>\d+)\t(?<dst_ip>\S+)\t(?<dst_port>\d+)\t(?<r>\S+)"
| where result="failure"
| stats count AS failed_attempts BY src_ip
| where failed_attempts > 3
| sort -failed_attempts
```

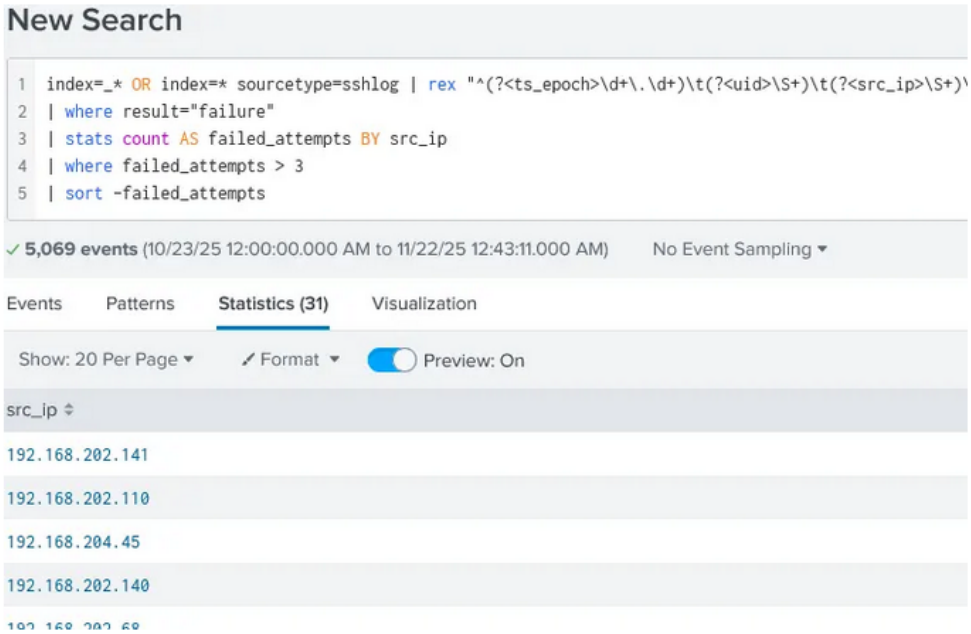

Figure 28: Source IPs with more than 3 failed SSH login attempts

13.2 Detect Behaviour Consistent with Brute-Force Attacks

Combine attempt volume, failure count, time window, and attack rate into a single query to produce a comprehensive brute-force indicator:

```
index=_* OR index=* sourcetype=sshlog
| rex
  "^(?<ts_epoch>\d+\.\d+)\t(?<uid>\S+)\t(?<src_ip>\S+)\t
   (?<src_port>\d+)\t(?<dst_ip>\S+)\t(?<dst_port>\d+)\t(?<r>\S+)"
| stats
    count                          AS attempts
    sum(eval(result="failed"))     AS failed
    earliest(_time)                AS first
    latest(_time)                  AS last
  BY src_ip
| eval attack_rate = attempts / (last - first)
| where failed > 20 OR attack_rate > 2
| eval duration = tostring(last - first, "duration")
| sort -attack_rate
```

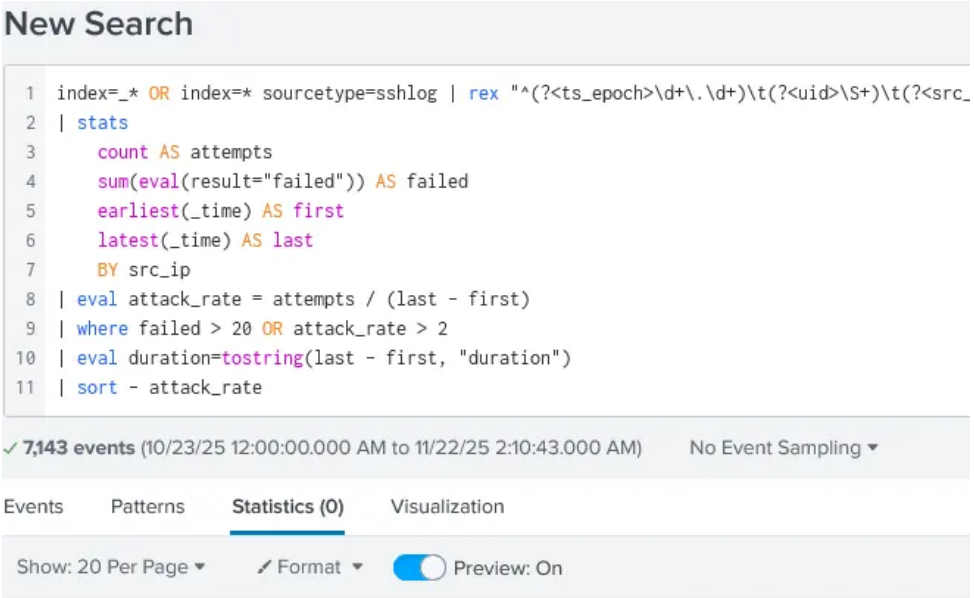

Figure 29: Comprehensive brute-force behaviour detection sorted by attack rate

Query Logic: This query flags a source IP if it has more than 20 failures *or* an attack rate exceeding 2 attempts per second. The `duration` field shows the total time window of the attack in human-readable form, making it easy to assess the persistence of each offending host.

Conclusion

Analyzing SSH log files using Splunk SIEM provides valuable insights into remote access activity across servers within a network. By monitoring SSH events, detecting anomalies, and correlating findings with other log sources, organizations can significantly enhance their security posture and protect against unauthorized access and potential security threats.

Note: The procedures and SPL queries demonstrated in this project can be customized to align with your specific operational environment, compliance framework, and organizational security requirements.

Technique	**Purpose**
`rex` Field Extraction	Parse structured fields (IP, port, result, version) from raw SSH log lines.
`stats count by result`	Compare successful vs. failed SSH authentication attempts.
`timechart span=5m`	Visualise SSH failure spikes over time to detect burst attack campaigns.
`eventstats` + `stdev`	Statistically flag rare source IPs that deviate from normal traffic volume.
Failure Threshold (`> 20`)	Identify brute-force candidates based on raw failure count per source IP.
Attack Rate (`attempts / time`)	Measure the velocity of connection attempts to distinguish automated tools from manual access.

Original post by Hassen Hannachi on Medium:
https://medium.com/@hassen-hannachi/analyzing-ssh-log-files-using-splunk-siem-73d3e29db8d3